

Local RAG Project: Learning Summary

Notes from building a fully local Retrieval-Augmented Generation pipeline

1. Local RAG is fully doable without cloud

A complete RAG pipeline can be built on a laptop using free, local tools: sentence-transformers for embeddings, FAISS for vector storage, and Ollama for running an LLM (Phi-3) locally. No API keys or cloud subscriptions are required.

2. Environment setup lessons

A real dependency conflict came up: upgrading numpy to 2.x (required by faiss) broke scipy and tensorflow, which needed numpy under 2.0. This reinforced a lesson from a previous project: always isolate project dependencies inside a virtual environment (.venv) instead of installing into the global Python environment.

3. Training vs. inference

Large labs (OpenAI, Anthropic, Meta, Google) train foundation models using massive datasets, custom distributed training infrastructure, and thousands of GPUs running for months. This is a completely separate process from what a RAG project does. A RAG pipeline only uses inference: running an already-trained model to generate outputs. No training happens in the pipeline itself.

4. What LangChain actually is

LangChain is not RAG itself. RAG is a technique: retrieve relevant context, then generate an answer using it. LangChain is a framework that provides pre-built components (document loaders, text splitters, vector store wrappers, chains, agents) that make it faster to build RAG or other LLM workflows, instead of writing every piece by hand.

5. Tool use and web search

LLMs cannot browse the internet on their own. To let a model search the web, it is given a described tool (e.g. a `web_search()` function). The model requests a call to that tool when needed, the surrounding code executes the actual search, and the results are fed back into the model as new context. This is the same underlying pattern as RAG, just with a live search API instead of a static FAISS index.

6. What a vector store actually is

A vector store is not a dictionary containing vectors for every word in existence. It only stores vectors for documents explicitly added to it. The embedding model is what knows how to convert any text into a vector; the vector store simply holds and searches the vectors that have been generated from your own documents.

7. The full RAG pipeline, step by step

Documents are split into chunks first (not embedded as a whole document). Each chunk is embedded individually, producing one vector per chunk. All chunk-vectors are stored in FAISS. A user's query is embedded the same way, using the same model. FAISS finds the closest stored chunks by vector distance. Those retrieved chunks are inserted into a prompt template, which is sent to the LLM. The LLM only ever sees the final assembled text prompt, it has no direct awareness of vectors, FAISS, or the retrieval process happening behind the scenes.

- 1 PDFs -> split into chunks
- 2 Each chunk -> embedding model -> vector (e.g. length 384)
- 3 All chunk-vectors -> stored in FAISS
- 4 User query -> embedded using the same model
- 5 FAISS -> finds closest chunk-vectors to the query
- 6 Retrieved chunks + query -> inserted into a prompt template
- 7 Prompt -> sent to the LLM
- 8 LLM -> generates the final answer

8. Worked example: query to answer

Five example sentences were stored in FAISS. When the query "What does SHAP do?" was embedded and compared against the stored vectors, the two closest matches were retrieved:

Rank	Retrieved chunk
1 (closest)	SHAP values explain individual predictions by attributing contribution to each feature.
2	XGBoost is a gradient boosting framework that uses decision trees as base learners.

Those two chunks were then assembled into a single prompt:

```
Answer the question using only the context below.
```

```
Context:
```

```
SHAP values explain individual predictions by attributing contribution to each feature.  
XGBoost is a gradient boosting framework that uses decision trees as base learners.
```

```
Question: What does SHAP do?
```

```
Answer:
```

This full block of text is what was sent to the LLM (Phi-3, via Ollama). The model never sees the original documents or the vectors, only this final assembled prompt. This separation is why RAG works with any LLM: swapping Phi-3 for Llama or GPT-4 would not change the retrieval half at all.

9. Key vocabulary

Term	Meaning
Inference	Using an already-trained model to generate predictions or outputs, as opposed to training.

Embedding	A numeric vector representation of text that captures its meaning, positioned so similar meanings
Agentic	Describing an AI system that can autonomously decide which actions or tools to use to complete
Ubiquitous	Present, appearing, or found everywhere.

10. Next step

Move from the toy 5-sentence example to real PDFs. The main new piece to implement is chunking, splitting real documents into smaller pieces before embedding them.